MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE, ALWAR

NAME OF FACULTY: RAJESH KUMAR

SUB & CODE: Database Management Systems (4CS4-04, 4AD4-04, 4AM4-04)

SEMESTER: IV | SESSION: 2023-24 (Even SEM.)

BRANCH: CS & AI/DS | BATCH: A & B

UNIT: IV | SUBMISSON: 15-3-2024

Lecture No.	Topics Covered	Page No.
1	Introduction State: Transaction State	1-2
2	Transaction Properties	3-4
3	Concurrent excecutions: Need of seriaziblity	5-9
4	Conflict Vs View seriaziblity, Testing for seriaziblity	10-14
5	Recoverable Schedules	15-17
6	Cascade less Schedules	18-21

References:

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F Korth
2. Fundamentals of Database Systems by Elmsari, Navathe, 5th Edition Pearson

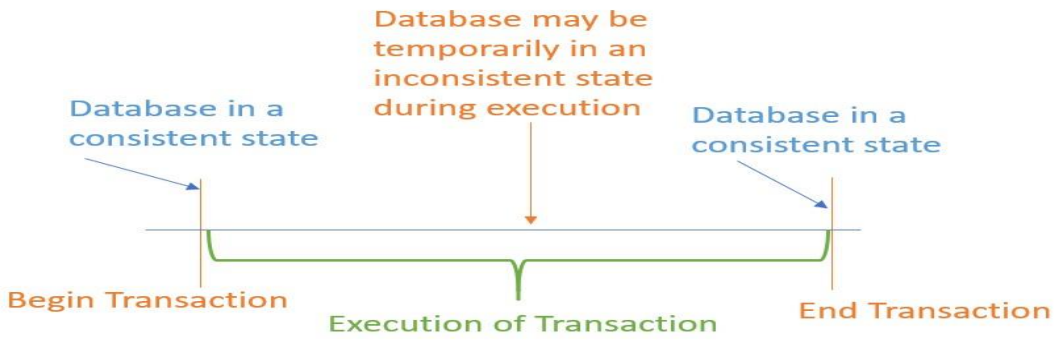
Faculty HOD Dean Academics 16/3/24

UNIT-4 (Introduction-Transaction State)
LECTURE NO: -27

What is a Database Transaction?

A Database Transaction is a logical unit of processing in a DBMS which entails one or more database access operation. Database transactions represent real-world events of any enterprise.

All types of database access operation which are held between the beginning and end transaction statements are considered as a single logical transaction in DBMS. During the transaction the database is inconsistent. Only once the database is committed the state is changed from one consistent state to another.



Database Transaction

Facts about Database Transactions

- A transaction is a program unit whose execution may or may not change the contents of a database.
- The transaction concept in DBMS is executed as a single unit.
- If the database operations do not update the database but only retrieve data, this type of transaction is called a read-only transaction.
- A successful transaction can change the database from one CONSISTENT STATE to another
- DBMS transactions must be atomic, consistent, isolated and durable

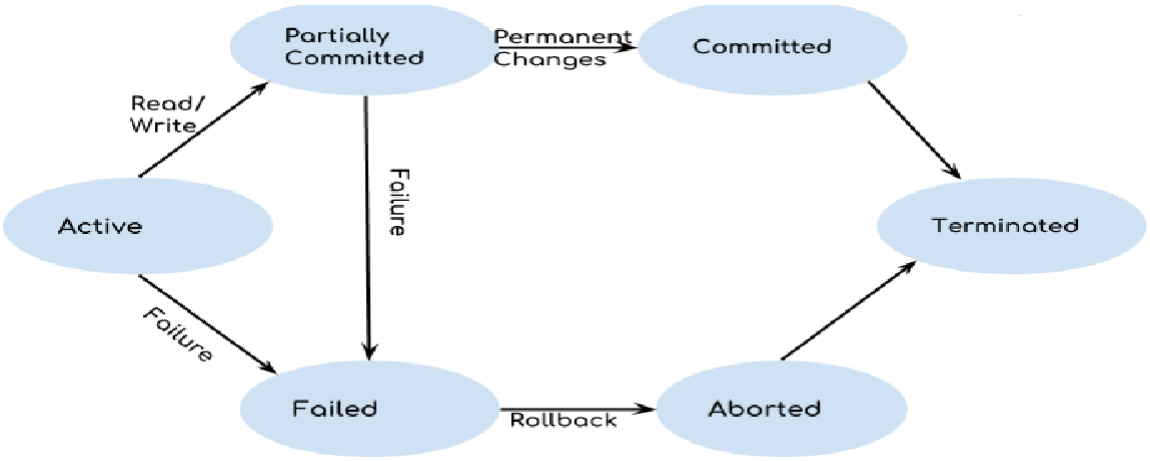
Why do you need concurrency in Transactions?

A database is a shared resource accessed. It is used by many users and processes concurrently. For example, the banking system, railway, and air reservations systems, stock market monitoring, supermarket inventory, and checkouts, etc.

A transaction is an action or series of actions that are being performed by a single user or application program, which reads or updates the contents of the database.

States of Transaction

In a database, the transaction can be in one of the following states –



Active State

As we have discussed in the DBMS transaction introduction that a transaction is a sequence of operations. If a transaction is in execution then it is said to be in active state. It doesn't matter which step is in execution, until unless the transaction is executing, it remains in active state.

Failed State

If a transaction is executing and a failure occurs, either a hardware failure or a software failure then the transaction goes into failed state from the active state.

Partially Committed State

a transaction goes into "partially committed" state from the active state when there are read and write operations present in the transaction.

A transaction contains number of read and write operations. Once the whole transaction is successfully executed, the transaction goes into partially committed state where we have all the read and write operations performed on the main memory (local memory) instead of the actual database.

The reason why we have this state is because a transaction can fail during execution so if we are making the changes in the actual database instead of local memory, database may be left in an inconsistent state in case of any failure.

Committed State

If a transaction completes the execution successfully then all the changes made in the local memory during partially committed state are permanently stored in the database. You can also see in the above diagram that a transaction goes from partially committed state to committed state when everything is successful.

Aborted State

if a transaction fails during execution then the transaction goes into a failed state. The changes made into the local memory (or buffer) are rolled back to the previous consistent state and the transaction goes into aborted state from the failed state. Refer the diagram to see the interaction between failed and aborted state.

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
3. C.J. Date: Data Base Design, Addison Wesley

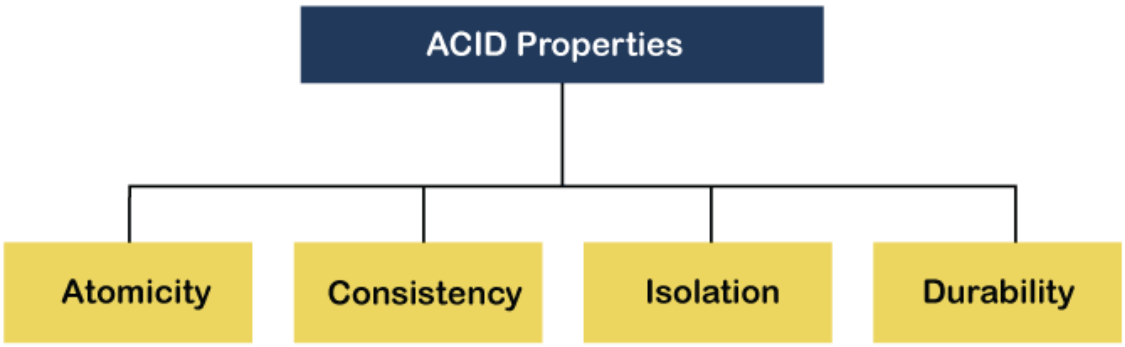
MITRC

UNIT-4 (Transaction properties)
LECTURE NO: -28

ACID Properties in DBMS

DBMS is the management of data that should remain integrated when any changes are done in it. It is because if the integrity of the data is affected, whole data will get disturbed and corrupted.

Therefore, to maintain the integrity of the data, there are four properties described in the database management system, which are known as the ACID properties. The ACID properties are meant for the transaction that goes through a different group of tasks.



Atomicity: -

This property states that a transaction must be treated as an atomic unit, that is, either all of its operations are executed or none. There must be no state in a database where a transaction is left partially completed. States should be defined either before the execution of the transaction or after the execution/abortion/failure of the transaction.

Consistency

Data is in a consistent state when a transaction starts and when it ends.

For example, in an application that transfers funds from one account to another, the consistency property ensures that the total value of funds in both the accounts is the same at the start and end of each transaction.

Isolation

In a database system where more than one transaction are being executed simultaneously and in parallel, the property of isolation states that all the transactions will be carried out and executed as if it is the only transaction in the system. No transaction will affect the existence of any other transaction.

Durability

In DBMS, the term durability ensures that the data after the successful execution of the operation becomes permanent in the database. The durability of the data should be so perfect that even if the system fails or leads to a crash, the database still survives. However, if gets lost, it becomes the responsibility of the recovery manager for ensuring the durability of the database. For committing the values, the COMMIT command must be used every time we make changes.

Therefore, the ACID property of DBMS plays a vital role in maintaining the consistency and availability of data in the database.

Operations in Transaction-

The main operations in a transaction are-

Read Operation

Write Operation

1. Read Operation

Read operation reads the data from the database and then stores it in the buffer in main memory.

For example- Read(A) instruction will read the value of A from the database and will store it in the buffer in main memory.

2. Write Operation

Write operation writes the updated data value back to the database from the buffer.

For example- Write(A) will write the updated value of A from the buffer to the database.

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
3. C.J. Date: Data Base Design, Addison Wesley

MITRC

UNIT-4 (Concurrent Executions & Need of Serializability)

LECTURE NO: -29

Concurrent Execution in DBMS

In a multi-user system, multiple users can access and use the same database at one time, which is known as the concurrent execution of the database. It means that the same database is executed simultaneously on a multi-user system by different users.

While working on the database transactions, there occurs the requirement of using the database by multiple users for performing different operations, and in that case, concurrent execution of the database is performed.

The thing is that the simultaneous execution that is performed should be done in an proper manner and no operation should affect the other executing operations, thus maintaining the consistency of the database.

Thus, on making the concurrent execution of the transaction operations, there occur several challenging problems that need to be solved.

Problems with Concurrent Execution

In a database transaction, the two main operations are READ and WRITE operations. So, there is a need to manage these two operations in the concurrent execution of the transactions as if these operations are not performed in an proper manner, and the data may become inconsistent.

1. Lost update problem (Write – Write conflict)

This type of problem occurs when two transactions in database access the same data item and have their operations in an interleaved manner (i.e., concurrent execution) that makes the value of the items incorrect hence making the database inconsistent.

Dirty Read Problems (W-R Conflict)

The dirty read problem occurs when one transaction updates an item of the database, and somehow the transaction fails, and before the data gets rollback, the updated database item is accessed by another transaction. There comes the Read-Write Conflict between both transactions.

Unrepeatable Read Problem (W-R Conflict)

Also known as Inconsistent Retrievals Problem that occurs when in a transaction, two different values are read for the same database item.

MITRC

Concurrent Executions & Need of Serializability

When multiple transactions are running concurrently then there is a possibility that the database may be left in an inconsistent state. Serializability is a concept that helps us to check which schedules are serializable. A serializable schedule is the one that always leaves the database in consistent state.

What is serializability?

- Serializability is a concurrency scheme where the concurrent transaction is equivalent to one that executes the transactions serially. A schedule is a list of transactions.
- serializability is a way to check if the execution of two or more transactions are maintaining the database consistency or not.
- The main objective of serializability is to find non-serial schedules that allow transactions to execute concurrently without interference and produce a database state that could be produced by a serial execution.

Schedules in DBMS are of two types:

- Serial schedule defines each transaction is executed consecutively without any interference from other transactions. In the serial schedule, when the first transaction completes its cycle, then the next transaction is executed.

Transaction-1	Transaction-2
R(a)	
W(a)	
R(b)	
W(b)	
	R(b)
	W(b)
	R(a)
	W(a)

NOTE: Serial schedules are always serializable because the transactions only work one after the other. Also, for a transaction, there are n! serial schedules possible (where n is the number of transactions).

- Non-serial schedule defines the operations from a group of concurrent transactions that are interleaved. It contains many possible orders in which the system can execute the individual operations of the transactions.

MITRC

Concurrent Executions & Need of Serializability

Transaction-1	Transaction-2
R(a)	
W(a)	
	R(b)
	W(b)
R(b)	
	R(a)
W(b)	
	W(a)

We can see that Transaction-2 starts its execution before the completion of Transaction-1, and they are interchangeably working on the same data, i.e., "a" and "b".

Types of Serializability

1. Conflict Serializability

2. View Serializability

MITRC

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
2. Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

UNIT-4 (Conflict vs View Serializability, Testing for Serializability)

LECTURE NO: -30

What is serializability?

- Serializability is a concurrency scheme where the concurrent transaction is equivalent to one that executes the transactions serially.
- A schedule is a list of transactions.
- Serial schedule defines each transaction is executed consecutively without any interference from other transactions. In the serial schedule, when the first transaction completes its cycle, then the next transaction is executed.
- Non-serial schedule defines the operations from a group of concurrent transactions that are interleaved. It contains many possible orders in which the system can execute the individual operations of the transactions.
- The main objective of serializability is to find non-serial schedules that allow transactions to execute concurrently without interference and produce a database state that could be produced by a serial execution.

Types of Serializability

1. Conflict Serializability
2. View Serializability

1. Conflict Serializability

- Conflict serializability defines two instructions of two different transactions accessing the same data item to perform a read/write operation.
- It deals with detecting the instructions that are conflicting in any way and specifying the order in which the instructions should execute in case there is any conflict.
- A conflict serializability arises when one of the instructions is a write operation.

MITRC

Conflict vs View Serializability, Testing for Serializability

The following rules are important in Conflict Serializability,

1. If two transactions are both read operation, then they are not in conflict.

2. If one transaction wants to perform a read operation and other transaction wants to perform a write operation, then they are in conflict and cannot be swapped.

3. If both the transactions are for write operation, then they are in conflict, but can be allowed to take place in any order, because the transactions do not read the value updated by each other.

Readi(x) readj(x) - non conflict read-read operation

Readi(x) writej(x) - conflict read-write operation.

Writei(x) readj(x) - conflict write-read operation.

Writei(x) writej(x) - conflict write-write operation.

2. View Serializability

1. View serializability is another type of serializability.
2. It can be derived by creating another schedule out of an existing schedule and involves the same set of transactions.

Example: Let us assume two transactions T1 and T2 that are being serialized to create two different schedules SH1 and SH2, where T1 and T2 want to access the same data item. Now there can be three scenarios

1. If in SH1, T1 reads the initial value of data item, then in SH2 , T1 should read the initial value of that same data item.

2. If in SH2, T1 writes a value in the data item which is read by T2, then in SH2, T1 should write the value in the data item before T2 reads it.

3. If in SH1, T1 performs the final write operation on that data item, then in SH2, T1 should perform the final write operation on that data item.

If a concurrent schedule is view equivalent to a serial schedule of same transaction then it is said to be View serializable.

MTRC

Conflict vs View Serializability, Testing for Serializability

S.No.	Conflict Serializability	View Serializability
1.	Two schedules are said to be conflict equivalent if all the conflicting operations in both the schedule get executed in the same order. If a schedule is a conflict equivalent to its serial schedule then it is called Conflict Serializable Schedule.	Two schedules are said to be view equivalent if the order of initial read, final write and update operations is the same in both the schedules. If a schedule is view equivalent to its serial schedule then it is called View Serializable Schedule.
2.	If a schedule is view serializable then it may or may not be conflict serializable.	If a schedule is conflict serializable then it is also view serializable schedule.
3.	Conflict equivalence can be easily achieved by reordering the operations of two transactions therefore, Conflict Serializability is easy to achieve.	View equivalence is rather difficult to achieve as both transactions should perform similar actions in a similar manner. Thus, View Serializability is difficult to achieve.
4.	For a transaction T1 writing a value A that no one else reads but later some other transactions say T2 write its own value of A, W(A) cannot be placed under positions where it is never read.	If a transaction T1 writes a value A that no other transaction reads (because later some other transactions say T2 writes its own value of A) W(A) can be placed in positions of the schedule where it is never read.

Testing of Serializability

Serialization Graph is used to test the Serializability of a schedule.

Assume a schedule S. For S, we construct a graph known as precedence graph. This graph has a pair G = (V, E), where V consists a set of vertices, and E consists a set of edges. The set of vertices is used to contain all the transactions participating in the schedule.

The set of edges is used to contain all edges Ti ->Tj for which one of the three conditions holds:

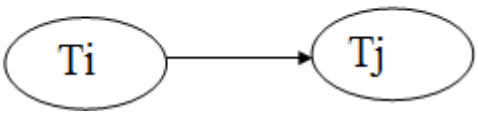
Conflict vs View Serializability, Testing for Serializability

Create a node Ti → Tj if Ti executes write (Q) before Tj executes read (Q).

Create a node Ti → Tj if Ti executes read (Q) before Tj executes write (Q).

Create a node Ti → Tj if Ti executes write (Q) before Tj executes write (Q).

Precedence graph for Schedule S



- If a precedence graph contains a single edge Ti → Tj, then all the instructions of Ti are executed before the first instruction of Tj is executed.
- If a precedence graph for schedule S contains a cycle, then S is non-serializable.
- If the precedence graph has no cycle, then S is known as serializable.

For example

Time

T1	T2	T3
Read(A)		
	Read(B)	
$A := f_1(A)$		
		Read(C)
	$B := f_2(B)$	
	Write(B)	
		$C := f_3(C)$
		Write(C)
Write(A)		
		Read(B)
	Read(A)	
	$A := f_4(A)$	
Read(C)		
	Write(A)	
$C := f_5(C)$		
Write(C)		
		$B := f_6(B)$
		Write(B)

Schedule S1

Testing for Serializability

Explanation:

Read(A): In T1, no subsequent writes to A, so no new edges

Read(B): In T2, no subsequent writes to B, so no new edges

Read(C): In T3, no subsequent writes to C, so no new edges

Write(B): B is subsequently read by T3, so add edge T2 → T3

Write(C): C is subsequently read by T1, so add edge T3 → T1

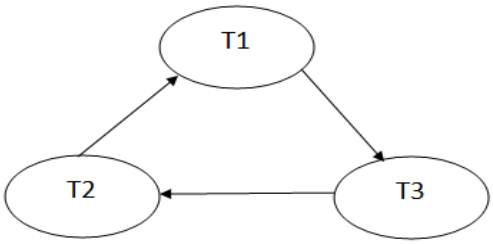
Write(A): A is subsequently read by T2, so add edge T1 → T2

Write(A): In T2, no subsequent reads to A, so no new edges

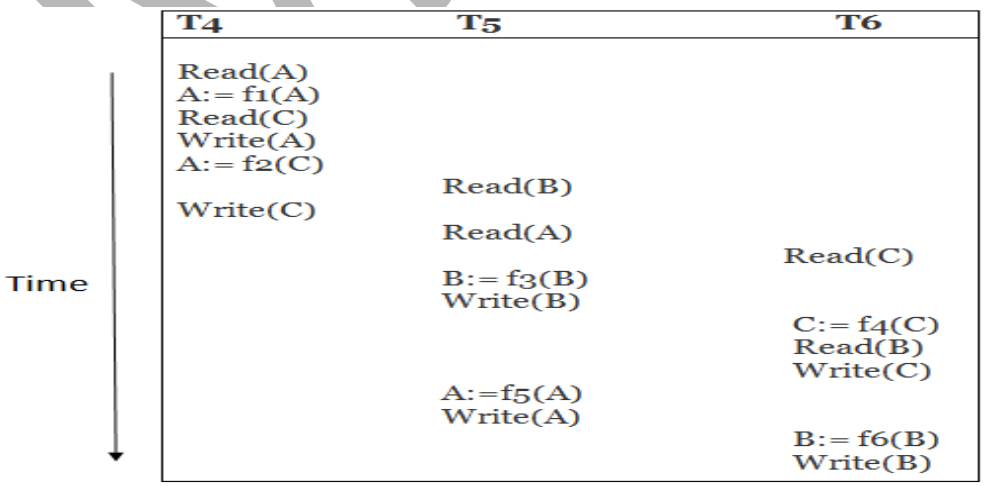
Write(C): In T1, no subsequent reads to C, so no new edges

Write(B): In T3, no subsequent reads to B, so no new edges

Precedence graph for schedule S1:



The precedence graph for schedule S1 contains a cycle that's why Schedule S1 is non-serializable.



Schedule S2

Explanation:

Read(A): In T4, no subsequent writes to A, so no new edges

Read(C): In T4, no subsequent writes to C, so no new edges

Write(A): A is subsequently read by T5, so add edge T4 → T5

Read(B): In T5, no subsequent writes to B, so no new edges

Write(C): C is subsequently read by T6, so add edge T4 → T6

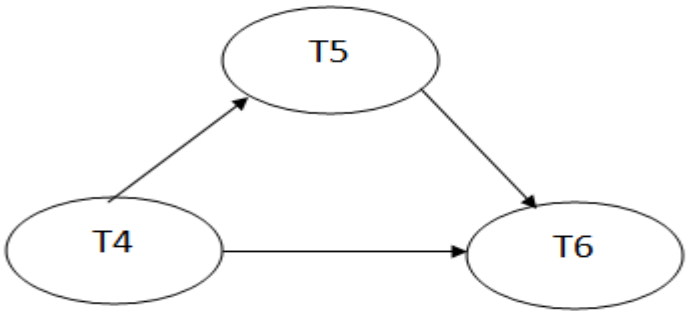
Write(B): A is subsequently read by T6, so add edge T5 → T6

Write(C): In T6, no subsequent reads to C, so no new edges

Write(A): In T5, no subsequent reads to A, so no new edges

Write(B): In T6, no subsequent reads to B, so no new edges

Precedence graph for schedule S2:

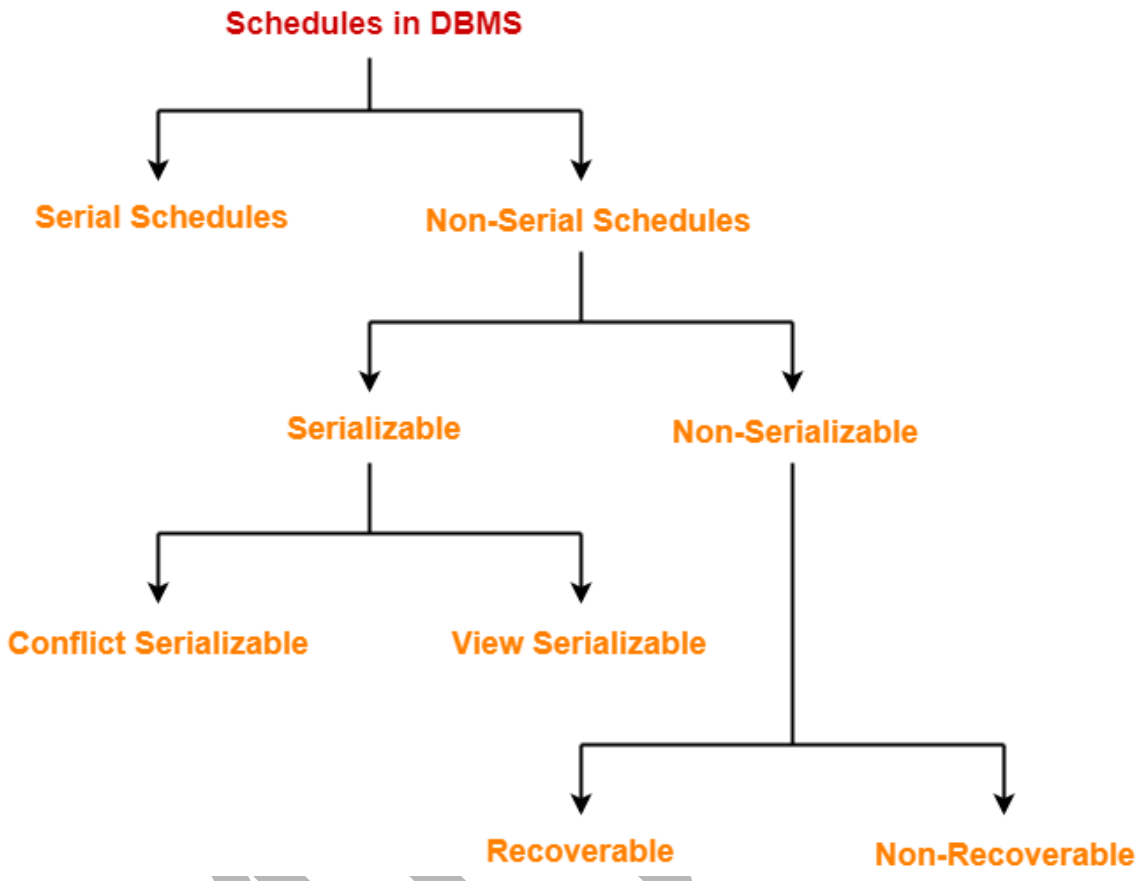


The precedence graph for schedule S2 contains no cycle that's why ScheduleS2 is serializable.

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
2. Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

UNIT-4 (Recoverable Schedules)
LECTURE NO: -31



Schedule

A series of operation from one transaction to another transaction is known as schedule. It is used to preserve the order of the operation in each of the individual transaction.

1. Serial Schedule

The serial schedule is a type of schedule where one transaction is executed completely before starting another transaction. In the serial schedule, when the first transaction completes its cycle, then the next transaction is executed.

2. Non-serial Schedule

If interleaving of operations is allowed, then there will be non-serial schedule.

It contains many possible orders in which the system can execute the individual operations of the transactions.

Serializable and Non Serializable schedule

The serializability of schedules is used to find non-serial schedules that allow the transaction to execute concurrently without interfering with one another. It identifies which schedules are correct when executions of the transaction have interleaving of their operations.

A non-serial schedule will be serializable if its result is equal to the result of its transactions executed serially.

(a)

Time

T1	T2

read(A);
A := A – N;
write(A);
read(B);
B := B + N;
write(B);
	read(A);
	A := A + M;
	write(A);

Schedule A

(b)

Time

T1	T2

read(A);
A:= A - N;
write(A);
read(B);
B:= B +N;
write(B);
	read(A);
	A := A + M;
	write(A);

Schedule B

(c)

Time

T1	T2
read(A); A := A – N;	
	read(A); A := A + M;
write(A); read(B);	
	write(A);
B := B + N; write(B);	

Schedule C

(d)

Time

T1	T2

read(A);
A := A – N;
write(A);	

	read(A);
	A := A + M;
	write(A);

read(B);
B := B + N;
write(B);	

Schedule D

Schedule A and Schedule B are serial schedule.

Schedule C and Schedule D are Non-serial schedule.

Recoverable Schedules

1. **Conflict Serializability**

- Conflict serializability defines two instructions of two different transactions accessing the same data item to perform a read/write operation.
- It deals with detecting the instructions that are conflicting in any way and specifying the order in which the instructions should execute in case there is any conflict.
- A conflict serializability arises when one of the instructions is a write operation.

2. View Serializability

- View serializability is another type of serializability.
- It can be derived by creating another schedule out of an existing schedule and involves the same set of transactions

Recoverable Schedule

If in a schedule,

- A transaction performs a dirty read operation from an uncommitted transaction
- And its commit operation is delayed till the uncommitted transaction either commits or roll backs

then such a schedule is known as a Recoverable Schedule.

Example

Let us consider two transaction schedules as given below −

T1	T2
Read(A)	
Write(A)	
-	Read(A) ///Dirty Read
-	Write(A)

-	
-	
Commit	
	Commit // delayed

The above schedule is a recoverable schedule because of the reasons mentioned below-

- The transaction T2 performs dirty read operation on A.
- The commit operation of transaction T2 is delayed until transaction T1 commits or rollback.
- Transaction commits later.
- In the above schedule transaction T2 is now allowed to commit whereas T1 is not yet committed.
- In this case transaction T1 is failed, and transaction T2 still has a chance to recover by rollback.

Irrecoverable Schedules-

If in a schedule,

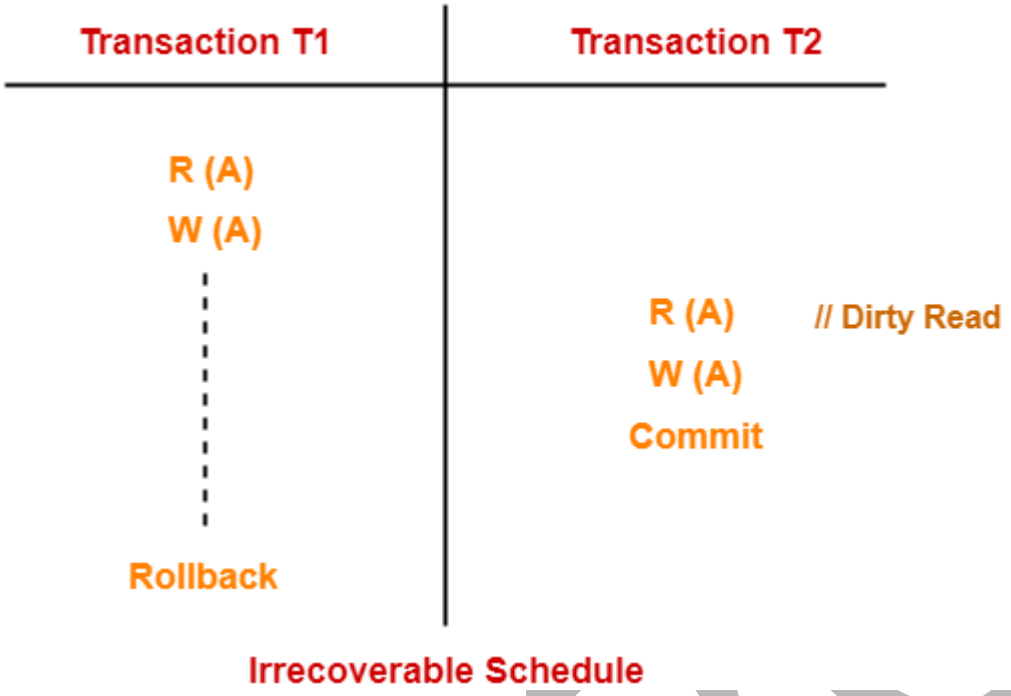
- A transaction performs a dirty read operation from an uncommitted transaction
- And commits before the transaction from which it has read the value

then such a schedule is known as an Irrecoverable Schedule.

Example-

Consider the following schedule-

MITRC



Irrecoverable Schedule

Here

- T2 performs a dirty read operation.
- T2 commits before T1.
- T1 fails later and roll backs.
- The value that T2 read now stands to be incorrect.
- T2 can not recover since it has already committed.

Problem: - Checking Whether a Schedule is Recoverable or Irrecoverable-

Method-01:

Check whether the given schedule is conflict serializable or not.

- If the given schedule is conflict serializable, then it is surely recoverable. Stop and report your answer.
- If the given schedule is not conflict serializable, then it may or may not be recoverable. Go and check using other methods.

Thumb Rules

All conflict serializable schedules are recoverable.

All recoverable schedules may or may not be conflict serializable.

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
2. Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

MITRC

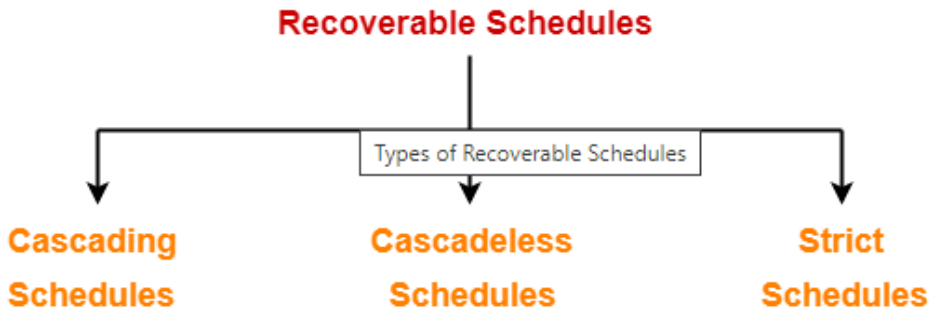
UNIT-4 (Cascadeless Schedules)
LECTURE NO: -32

Recoverable Schedules-

If in a schedule,

- A transaction performs a dirty read operation from an uncommitted transaction
- And its commit operation is delayed till the uncommitted transaction either commits or roll backs

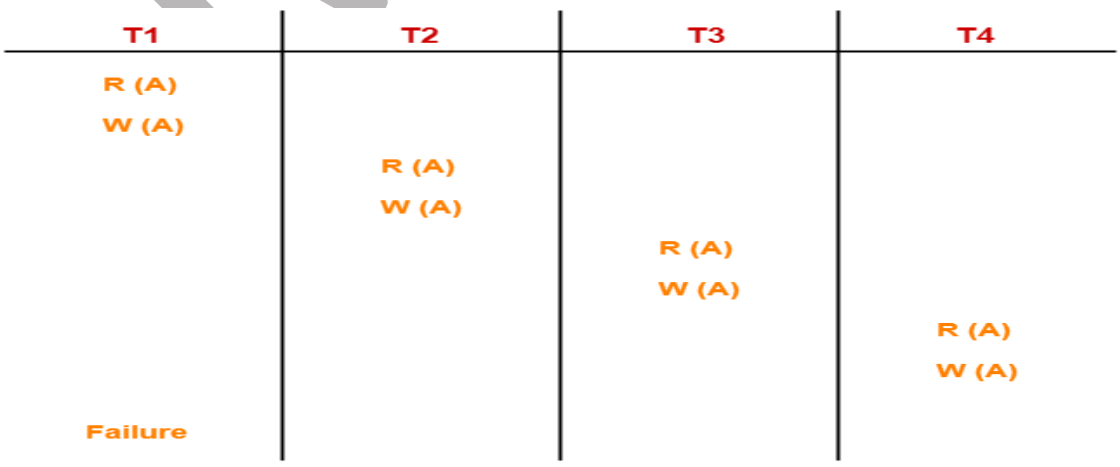
then such a schedule is called as a Recoverable Schedule.



Cascading Schedule-

- If in a schedule, failure of one transaction causes several other dependent transactions to rollback or abort, then such a schedule is called as a Cascading Schedule or Cascading Rollback .
- It simply leads to the wastage of CPU time.

Example-



Cascading Recoverable Schedule

Here,

- Transaction T2 depends on transaction T1.
- Transaction T3 depends on transaction T2.
- Transaction T4 depends on transaction T3.

In this schedule,

- The failure of transaction T1 causes the transaction T2 to rollback.
- The rollback of transaction T2 causes the transaction T3 to rollback.
- The rollback of transaction T3 causes the transaction T4 to rollback.

  Such a rollback is called as a Cascading Rollback.

NOTE-

If the transactions T2, T3 and T4 would have committed before the failure of transaction T1, then the schedule would have been irrecoverable.

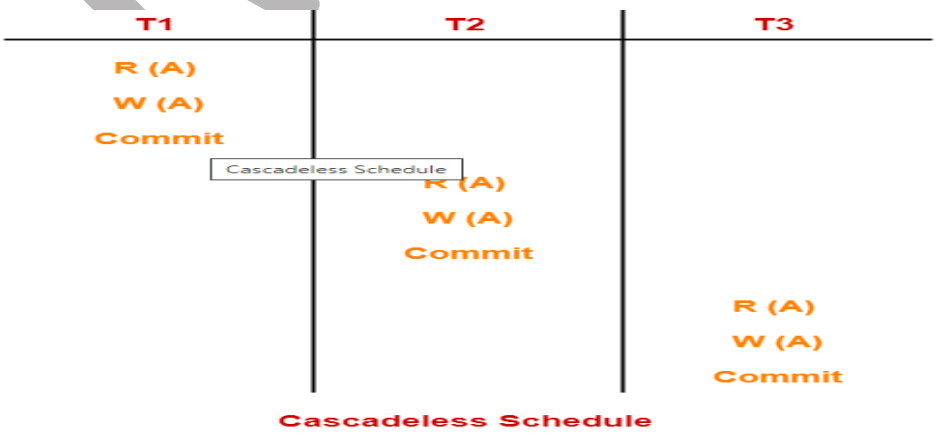
Cascadeless Schedule-

If in a schedule, a transaction is not allowed to read a data item until the last transaction that has written it is committed or aborted, then such a schedule is called as a Cascadeless Schedule.

In other words,

Cascadeless schedule allows only committed read operations.

Therefore, it avoids cascading roll back and thus saves CPU time.



Cascadeless Schedules

NOTE-

Cascadeless schedule allows only committed read operations.

However, it allows uncommitted write operations.

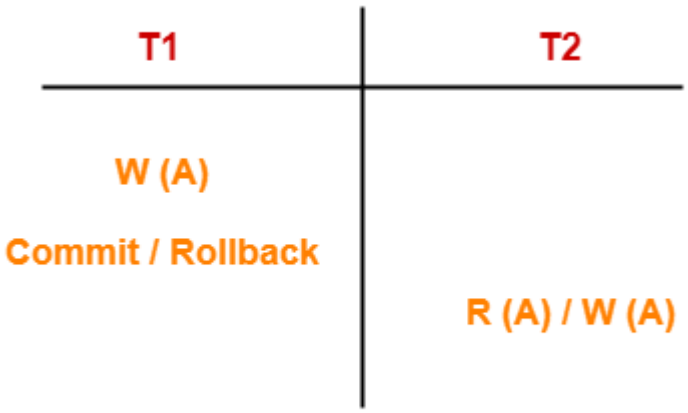
Strict Schedule-

If in a schedule, a transaction is neither allowed to read nor write a data item until the last transaction that has written it is committed or aborted, then such a schedule is called as a Strict Schedule.

In other words,

Strict schedule allows only committed read and write operations.

Clearly, strict schedule implements more restrictions than cascadeless schedule.



Strict Schedule

Remember-

Strict schedules are more strict than cascadeless schedules.

All strict schedules are cascadeless schedules.

All cascadeless schedules are not strict schedules.

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
2. Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).